

REUSABLE ARRAYS, STRINGS & HASHING UTILITIES

Array & String Conversions:

```
// int[] → List
List<Integer> list =
Arrays.stream(arr).boxed().collect(Collectors.toList());
// List → int[]
int[] arr = list.stream().mapToInt(i → i).toArray();
// String → char[] → String
char[] chars = str.toCharArray();
String s = new String(chars);
```

HashMap & Frequency Utilities:

```
// Character frequency array (a-z)
int[] freq = new int[26];
freq[ch - 'a']++;
// HashMap getOrDefault & computeIfAbsent
map.put(key, map.getOrDefault(key, 0) + 1);
map.computeIfAbsent(key, k → new ArrayList<>()).add(val);
```

TOPIC 02: ARRAYS & STRINGS FOUNDATION

AHA MOMENT #1: THE MEMORY VS SPEED TRADEOFF
The interviewer isn't asking for a HashMap or HashSet by name. They are asking: "Can you trade auxiliary space complexity to reduce runtime from $O(N^2)$ to $O(N)$?"
Data structures are simply physical vehicles for this memory-speed trade!

1 ARRAY MEMORY & OFFSET MATHEMATICS

Arrays are contiguous memory blocks. Element address is computed in $O(1)$ time using the offset formula:

$$\text{Address}(\text{arr}[i]) = \text{Base_Address} + (i * \text{Element_Size_Bytes})$$

- **Access / Modify:** $O(1)$ constant time.
- **Insert / Delete (Middle):** $O(N)$ due to element shifting.
- **Search (Unsorted):** $O(N)$ linear scan.

2 JAVA STRINGS & STRING POOL INTERNALS

Strings in Java are **immutable** objects backed by a `byte[]` array (Java 9+ compact strings).

- **String Pool:** Special memory area inside JVM Heap preventing duplicate string literal allocations.
- **Concatenation Trap:** `s += char` inside a loop creates N temporary objects, resulting in $O(N^2)$ time! Always use `StringBuilder` for $O(N)$ append operations.

ARRAY & STRING TIME/SPACE COMPLEXITY CHEAT SHEET

Operation	Array ('int[]')	ArrayList ('List')	String ('String')	StringBuilder
Index Lookup	$O(1)$	$O(1)$	$O(1)$ via <code>charAt()</code>	$O(1)$
Append (End)	N/A (Fixed size)	$O(1)$ amortized	$O(N)$ (New copy)	$O(1)$ amortized
Insert / Remove (Middle)	$O(N)$	$O(N)$ (Array copy)	$O(N)$	$O(N)$
Search Element	$O(N)$	$O(N)$	$O(N)$ via <code>indexOf()</code>	$O(N)$

KEY TAKEAWAY: Always use primitive arrays or frequency arrays (e.g. `int[26]`) when character set is bounded (a-z) to avoid HashMap object boxing overhead!

💡 **AHA MOMENT #2: TREEIFICATION AT BUCKET SIZE 8**

Java 8+ HashMaps prevent Denial-of-Service collision attacks by automatically converting a linked list bucket into a **Red-Black Tree** when bucket size reaches ≥ 8 and table capacity ≥ 64 . This caps worst-case search complexity at $O(\log N)$ instead of $O(N)$!

1 HASHMAP ARCHITECTURE & BUCKETS

A HashMap is an array of buckets (Node<K,V>[] table).

- `hashCode()` computes 32-bit integer.
- `hash()` applies bit mixing to spread high bits.
- Index calculation: `index = (n - 1) & hash` (Fast bitwise AND for power-of-two table size 2^n).

2 LOAD FACTOR & REHASHING

- Default Capacity:** 16 buckets.
- Load Factor:** 0.75 (Threshold = $16 * 0.75 = 12$).
- Rehashing:** When size > threshold, array doubles to 32. Every existing node is re-indexed. This costs $O(N)$ once, but amortizes to $O(1)$ per operation!

⚡ HASHMAP & HASHSET PERFORMANCE MATRIX

Operation	Average Time	Worst Case (Java 7 - LinkedList)	Worst Case (Java 8+ - Treeified)	Space Complexity
put(key, val)	$O(1)$	$O(N)$ (Collision chain)	$O(\log N)$ (Red-Black Tree)	$O(N)$
get(key)	$O(1)$	$O(N)$	$O(\log N)$	$O(N)$
containsKey(key)	$O(1)$	$O(N)$	$O(\log N)$	$O(N)$
remove(key)	$O(1)$	$O(N)$	$O(\log N)$	$O(N)$

⚠️ **JAVA INTERVIEW TRAP:** Always override BOTH `equals()` and `hashCode()` when using custom objects as HashMap keys! Failing to do so causes duplicate keys and lookup failures.

💡 AHA MOMENT #3: TRAIN INSTINCT BEFORE CODING

Do not jump into writing code! Senior engineers scan problem statements for specific trigger keywords that map directly to data structure choices within 5 seconds of reading the prompt.

🔑 KEYWORD-TO-PATTERN DIRECTORY

Problem Keyword / Signal	Target Instinct Pattern	Data Structure Choice	Primary FAANG Example
"Find duplicate / Check uniqueness"	Set Lookup	HashSet<T>	Contains Duplicate (LC 217)
"Count occurrences / Character frequency"	Frequency Tracking	int[26] or HashMap<Character, Integer>	Valid Anagram (LC 242)
"Find pair with target sum"	Complement Map Lookup	HashMap<Value, Index>	Two Sum (LC 1)
"Group items by anagram / similarity"	Canonical Key Mapping	HashMap<CanonicalKey, List<T>>	Group Anagrams (LC 49)
"Find Top K Frequent elements"	Bucket Sort / Min-Heap	List<Integer>[] buckets or PriorityQueue	Top K Frequent Elements (LC 347)
"Longest consecutive sequence"	Set Sequence Boundary	HashSet<Integer>	Longest Consecutive Sequence (LC 128)

🔧 DEDICATED JAVA INTERVIEW HACKS & APIS

1. Frequency Defaulting (Cleaner Code):

```
// Avoid boilerplate if/else:
map.put(key, map.getOrDefault(key, 0) + 1);
```

2. Grouping List Initialization:

```
// Instantiates list if missing in 1 line:
map.computeIfAbsent(key, k → new ArrayList<>()).add(val);
```

🔑 KEY TAKEAWAY: Master `getOrDefault()` and `computeIfAbsent()` to write clean, bug-free Java solutions during live technical interviews!

1. LEETCODE 217 — CONTAINS DUPLICATE

Difficulty: Easy
Pattern: HashSet
Uniqueness
Companies: Apple, Amazon, Google
Freq: ★★★★★

Problem:

Given array

nums

, return

true

if any value appears at least twice.

Example:

nums = [1, 2, 3, 1]

→

true

.

Interviewer: Can you solve this without extra memory?

Candidate: Yes, by sorting first in $O(N \log N)$ time and $O(1)$ space. But using a HashSet gives optimal $O(N)$ time!

Complexity:

Time:

$O(N)$

, Space:

$O(N)$

.

```

public boolean containsDuplicate(int[] nums) {
    Set<Integer> seen = new HashSet<>();
    for (int num : nums) {
        // Step 1: Check if already present in Set
        if (seen.contains(num)) return true;
        // Step 2: Add to set for future lookups
        seen.add(num);
    }
    return false; // Step 3: All elements unique
}
        
```

2. LEETCODE 242 — VALID ANAGRAM

Difficulty: Easy
Pattern: Frequency Array
Companies: Uber, Google, Meta
Freq: ★★★★★

Problem:

Given strings

s

and

t

, return

true

if

t

is an anagram of

s

.

Example:

s = "anagram", t = "nagaram"

→

true

```

public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] count = new int[26];
    // Step 1: Increment for s, decrement for t
    for (int i = 0; i < s.length(); i++) {
        count[s.charAt(i) - 'a']++;
        count[t.charAt(i) - 'a']--;
    }
    // Step 2: Verify all counts zero
    for (int c : count) {
        if (c != 0) return false;
    }
    return true;
}
        
```

3. LEETCODE 1 — TWO SUM

Difficulty: Easy
 Pattern: Complement Map
 Companies: Google, Meta, Amazon, Apple
 Freq: ★★★★★

Problem:

Given array

nums

and

target

, return indices of two numbers that add to

target

.

Example:

nums = [2, 7, 11, 15], target = 9

→

[0, 1]

.

Evolution Strategy:

- Brute Force: $O(N^2)$ nested loops.
- Optimal: Store complement (`target - nums[i]`) in HashMap during a single pass!

```

public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        // Step 1: Check if complement exists in map
        if (map.containsKey(complement)) {
            return new int[]{map.get(complement), i};
        }
        // Step 2: Store current number and index
        map.put(nums[i], i);
    }
    return new int[0];
}

```

4. LEETCODE 49 — GROUP ANAGRAMS

Difficulty: Medium
 Pattern: Canonical Key Hashing
 Companies: Amazon, Meta, Uber
 Freq: ★★★★★

Problem:

Group array of strings

strs

into anagram lists.

Example:

["eat", "tea", "tan", "ate", "nat", "bat"]

→

[["bat"],["nat","tan"],["ate","eat","tea"]]

.

Canonical Key Trick:

Build a character frequency signature string (e.g. `#1#0#0 ... #2`) as HashMap key!

```

public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        int[] count = new int[26];
        for (char c : s.toCharArray()) count[c - 'a']++;
        // Step 1: Build canonical frequency key
        StringBuilder sb = new StringBuilder();
        for (int c : count) sb.append('#').append(c);
        String key = sb.toString();
        // Step 2: Group by key
        map.computeIfAbsent(key, k → new ArrayList<>()).add(s);
    }
    return new ArrayList<>(map.values());
}

```


AHA MOMENT (LC 49): Sorting each string takes $O(K \log K)$ per word. Generating a frequency tuple key takes $O(K)$ linear time per word!

5. LEETCODE 347 — TOP K FREQUENT ELEMENTS (BUCKET SORT $O(N)$)

Difficulty: Medium **Pattern:** Frequency Map + Bucket Sort **Companies:** Meta, Google, Amazon **Freq:** ★★★★★

Problem:

Return the

k

most frequent elements in

nums

.

Example:

nums = [1,1,1,2,2,3], k = 2

→

[1, 2]

.

Evolution Strategy:

- Min-Heap: $O(N \log K)$ time.
- Bucket Sort: $O(N)$ time by using array index as frequency count!

```
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> count = new HashMap<>();
    for (int n : nums) count.put(n, count.getOrDefault(n, 0) + 1);
    // Step 1: Array of buckets where index = frequency
    List<Integer>[] bucket = new List[nums.length + 1];
    for (int key : count.keySet()) {
        int freq = count.get(key);
        if (bucket[freq] == null) bucket[freq] = new ArrayList<>();
        bucket[freq].add(key);
    }
    // Step 2: Collect top K from highest bucket downwards
    int[] res = new int[k]; int idx = 0;
    for (int i = bucket.length - 1; i >= 0 && idx < k; i--) {
        if (bucket[i] != null) {
            for (int num : bucket[i]) { res[idx++] = num; if (idx == k) break; }
        }
    }
    return res;
}
```

6. LEETCODE 36 — VALID SUDOKU

Difficulty: Medium **Pattern:** HashSet Hash Matrix **Companies:** Apple, Uber, Meta **Freq:** ★★★★★

Problem:

Determine if a 9×9 Sudoku board is valid (Rows, Cols, 3×3 Sub-boxes).

```
public boolean isValidSudoku(char[][] board) {
    Set<String> seen = new HashSet<>();
    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {
            char val = board[r][c];
            if (val != '.') {
                if (!seen.add(val + " in row " + r) ||
                    !seen.add(val + " in col " + c) ||
                    !seen.add(val + " in box " + r/3 + "-" + c/3))
                    return false;
            }
        }
    }
    return true;
}
```

🔑 **KEY TAKEAWAY:** Bucket Sort achieves true linear $O(N)$ runtime for Top K Frequent elements by bounding frequency indices between 0 and N !

7. LEETCODE 128 — LONGEST CONSECUTIVE SEQUENCE

Difficulty: Medium **Pattern:** HashSet Sequence Start **Companies:** Google, Meta, Amazon **Freq:** ★★★★★

Problem:

Find length of longest consecutive elements sequence in unsorted array.

Example:

nums = [100, 4, 200, 1, 3, 2]

→

4

([1, 2, 3, 4]).

Sequence Start Trick:

Only start counting consecutive sequence if `!set.contains(num - 1)`! This ensures every number is visited at most twice ($O(N)$ time!).

```
public int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int num : nums) set.add(num);
    int longest = 0;
    for (int num : set) {
        // Step 1: Check if num is the start of a sequence
        if (!set.contains(num - 1)) {
            int currentNum = num;
            int currentStreak = 1;
            while (set.contains(currentNum + 1)) {
                currentNum++; currentStreak++;
            }
            longest = Math.max(longest, currentStreak);
        }
    }
    return longest;
}
```

8. LEETCODE 41 — FIRST MISSING POSITIVE (IN-PLACE HASHING)

Difficulty: Hard **Pattern:** Index Cyclic Sort **Companies:** Meta, Google, Amazon **Freq:** ★★★★★

Problem:

Find smallest missing positive integer in $O(N)$ time and $O(1)$ auxiliary space.

Example:

nums = [3, 4, -1, 1]

→

2

.

```
public int firstMissingPositive(int[] nums) {
    int n = nums.length;
    // Step 1: Place each number at index nums[i] - 1
    for (int i = 0; i < n; i++) {
        while (nums[i] > 0 && nums[i] ≤ n && nums[nums[i] - 1] ≠
nums[i]) {
            int temp = nums[nums[i] - 1];
            nums[nums[i] - 1] = nums[i];
            nums[i] = temp;
        }
    }
    // Step 2: First index where nums[i] ≠ i + 1 is missing
    for (int i = 0; i < n; i++) {
        if (nums[i] ≠ i + 1) return i + 1;
    }
    return n + 1;
}
```

 **AHA MOMENT (LC 128):** Without the `!set.contains(num - 1)` check, duplicate sequence scans would degrade execution time to $O(N^2)$!

9. LEETCODE 238 — PRODUCT OF ARRAY EXCEPT SELF

Difficulty: Medium**Pattern:** Prefix & Suffix Accumulation**Companies:** Amazon, Meta, Apple**Freq:** ★★★★★

Problem:

Return array

res

where

res[i]

equals product of all elements except

nums[i]

without division!

Example:

nums = [1, 2, 3, 4]

→

[24, 12, 8, 6]

.

```

public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    // Step 1: Prefix products
    res[0] = 1;
    for (int i = 1; i < n; i++) res[i] = res[i - 1] * nums[i - 1];
    // Step 2: Suffix products on the fly
    int right = 1;
    for (int i = n - 1; i ≥ 0; i--) {
        res[i] *= right;
        right *= nums[i];
    }
    return res;
}
        
```

10. LEETCODE 42 — TRAPPING RAIN WATER (PREFIX/SUFFIX MAX PREVIEW)

Difficulty: Hard**Pattern:** Prefix & Suffix Max Arrays**Companies:** Google, Meta, Amazon**Freq:** ★★★★★

Problem:

Compute total water trapped given elevation map

height

.

Formula:

water[i] = min(leftMax[i], rightMax[i]) - height[i]

.

```

public int trap(int[] height) {
    int n = height.length; if (n == 0) return 0;
    int[] leftMax = new int[n], rightMax = new int[n];
    leftMax[0] = height[0];
    for (int i = 1; i < n; i++) leftMax[i] = Math.max(leftMax[i - 1],
height[i]);
    rightMax[n - 1] = height[n - 1];
    for (int i = n - 2; i ≥ 0; i--) rightMax[i] =
Math.max(rightMax[i + 1], height[i]);
    int water = 0;
    for (int i = 0; i < n; i++) water += Math.min(leftMax[i],
rightMax[i]) - height[i];
    return water;
}
        
```

★ **TOPIC 03 TEASER:** Trapping Rain Water can be further optimized from $O(N)$ space down to $O(1)$ space using Two Pointers in Topic 03!

⚠️

TOP 10 COMMON ARRAYS & HASHING INTERVIEW MISTAKES

1. **String Concatenation in Loops:** Using `s += char` inside loops creates $O(N^2)$ memory copies. Use `StringBuilder`!

2. **Unnecessary HashMap Usage:** Allocating HashMap for 26 English letters instead of `int[26]` frequency array.

3. **Missing hashCode() Override:** Overriding `equals()` without `hashCode()` breaks HashMap lookups.

4. **Mutating Map Keys:** Modifying object properties while stored as a key inside a HashMap.

5. **Integer Overflow in Array Math:** Failing to use `long` when computing large array products or sums.

6. **Concurrent Modification Exception:** Removing elements from Map while iterating without Iterator.

7. **Off-By-One Array Access:** Accessing `arr[arr.length]` causing runtime `ArrayIndexOutOfBoundsException`.

8. **Assuming Unique Inputs:** Failing to handle duplicate elements in Two Sum or Group Anagrams.

9. **Ignoring Null / Empty Inputs:** Crashing on `null` or 0-length input array boundaries.

10. **Over-Allocating Initial Capacity:** Creating huge initial maps for small datasets causing memory bloat.

?

REVERSE QUESTIONS DECISION TREE ("WHY NOT?")

Alternative Choice	Why We Reject It In FAANG Interviews
Why Not Brute Force Nested Loops?	Exceeds $O(N^2)$ time limit for input size $N \geq 10^5$. Time Limit Exceeded (TLE).
Why Not Sort First?	Sorting takes $O(N \log N)$ time and destroys original array index positions needed for Two Sum.
Why Not TreeMap / TreeSet?	TreeMap operations take $O(\log N)$ per insertion vs HashMap's $O(1)$ constant time performance. Use TreeMap ONLY when sorted key order is explicitly required!

🧠

FAANG COACH TIP: Always explain to your interviewer WHY you chose a HashMap over sorting or brute force before typing your first line of code!

★ TOPIC 02 MASTER PATTERN RECONCILIATION MATRIX

Pattern	Core Data Structure	Time Complexity	Space Complexity	Representative LeetCode Problem
Set Uniqueness	HashSet<T>	O(N)	O(N)	LC 217 Contains Duplicate
Frequency Array	int[26] / int[128]	O(N)	O(1)	LC 242 Valid Anagram
Complement Map	HashMap<Val, Index>	O(N)	O(N)	LC 1 Two Sum
Canonical Key	HashMap<Key, List>	O(N · K)	O(N · K)	LC 49 Group Anagrams
Bucket Sort	List<Integer>[]	O(N)	O(N)	LC 347 Top K Frequent
Sequence Start	HashSet<Integer>	O(N)	O(N)	LC 128 Longest Consecutive

 NEETCODE & BLIND 75 PROGRESS TRACKER

- ☒ LC 217 Contains Duplicate
- ☒ LC 242 Valid Anagram
- ☒ LC 1 Two Sum
- ☒ LC 49 Group Anagrams
- ☒ LC 347 Top K Frequent
- ☒ LC 36 Valid Sudoku
- ☒ LC 128 Longest Consecutive
- ☒ LC 41 First Missing Positive
- ☒ LC 238 Product Except Self

✓ CHECKLIST BEFORE MOVING TO TOPIC 03 (TWO POINTERS)

- ☒ I understand HashMap collision resolution (Java 8 Red-Black Tree treeification at 8).
- ☒ I know when to use int[26] over HashMap for memory optimization.
- ☒ I can write Two Sum, Group Anagrams, and Top K Frequent from memory in <3 mins.

 TOPIC 02 COMPLETE! Turn the page to Topic 03: Two Pointers Masterclass!